Part 1 – Application Requirements and Business Rules

1.1 Problem and Motivation

The Game Rental Shop needs a database to record and track game rentals and returns for its members in a reliable, organized way. Manual, paper-based tracking is error-prone: staff can forget return dates, lose track of which games are currently rented out, or miscalculate rental revenue. A database lets the shop quickly check which games are rented, which members have not yet returned a game, and analyze revenue by publisher or genre.

1.2 Target Users

- Shop staff, who record rental and return transactions and maintain game/member information.
- The shop owner/manager, who reviews revenue reports and rental activity to make business decisions.
- Members who rent games, including individual customers and corporate clients (e.g., internet cafes, esports organizations, event companies).

1.3 Main Data

Game information (name, rental price, publisher, genre(s)); publisher information (name, country, year founded, target group); member information (name, phone number, membership tier, join date, referrer), split into individual members (ID card number) and corporate members (company name, tax ID); and rental transaction information (rental date, return date).

1.4 Business Rules

1. A publisher can publish many games, but each game is published by exactly one publisher.

2. A game can belong to many genres, and each genre can classify many games.

3. A member can rent many games, and a game can be rented by many members over time; each rental transaction records a rental date and a return date.

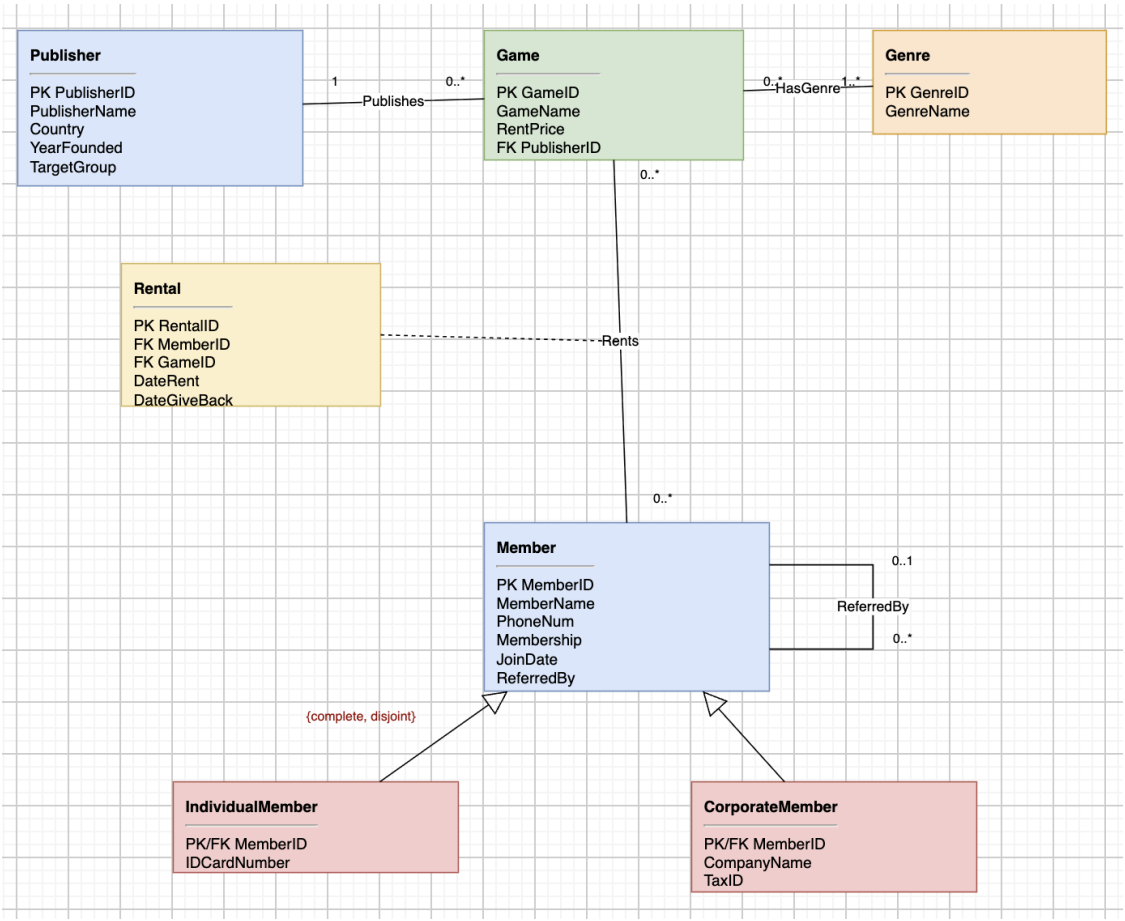
4. A member may be referred by at most one other member, but a member can refer many new members.

5. Every member must be classified as exactly one of two types: an individual member or a corporate member, never both.

6. Each member has a membership tier (Bronze, Silver, or Gold) and a join date that must always be recorded.

Part 2 – UML Class Diagram

The diagram below models the Game Rental Shop domain described in Part 1. It contains classes with attributes and identifiers, associations with multiplicity (1, 0..1, 0..*, 1..*), a many-to-many relationship (Game–Genre), an association class (Rental), and a generalization/specialization structure (Member / IndividualMember / CorporateMember).



Q1. Multiplicity

Association chosen: Publisher – Publishes – Game (multiplicity 1 on the Publisher side, 0..* on the Game side). Each Game must be published by exactly one Publisher (mandatory, single value). Each Publisher can publish zero or many Games — a newly registered publisher may not yet have any game in the system, while an established publisher such as Nintendo can have many games linked to it.

Q2. Association Class

The association class is Rental, attached to the many-to-many “Rents” association between Member and Game. Its attributes (RentalID, DateRent, DateGiveBack) describe one specific rental transaction. They cannot belong to Member alone, because a member's rental dates change every time a different game is rented; they cannot belong to Game alone, because a game's rental dates depend on who rents it and when. These attributes only make sense in the context of one particular Member–Game pairing at one point in time, which is exactly what an association class represents.

Q3. Generalization

The superclass Member is specialized into IndividualMember and CorporateMember. Every member shares common attributes (MemberID, MemberName, PhoneNum, Membership, JoinDate) but carries one additional set of attributes depending on type: individual members have an IDCardNumber, and corporate members have a CompanyName and TaxID. Modeling it this way keeps shared data in one place and avoids irrelevant, always-NULL columns that would appear if all attributes were merged into a single table (e.g., a TaxID column on a private customer). The {complete, disjoint} constraint means every member must belong to exactly one of the two subclasses — never neither, never both.

Part 3 – Relational Schema

The UML class diagram from Part 2 is translated into the following relational schema. Primary keys are marked PK and foreign keys are marked FK.

PUBLISHER(PublisherID PK, PublisherName, Country, YearFounded, TargetGroup)
GENRE(GenreID PK, GenreName)
GAME(GameID PK, GameName, RentPrice, PublisherID FK)
GAMEGENRE(GameID PK/FK, GenreID PK/FK)
MEMBER(MemberID PK, MemberName, PhoneNum, Membership, JoinDate, ReferredBy FK)
INDIVIDUALMEMBER(MemberID PK/FK, IDCardNumber)
CORPORATEMEMBER(MemberID PK/FK, CompanyName, TaxID)
RENTAL(RentalID PK, MemberID FK, GameID FK, DateRent, DateGiveBack)

Q4. One-to-many association → foreign key

The Publisher–Game association (1 : 0..*) is represented by placing a foreign key, PublisherID, in the GAME table, referencing PUBLISHER.PublisherID. Because each game has exactly one publisher, the foreign key is placed on the “many” side (Game) and declared NOT NULL; no separate table is needed since one FK column fully captures the relationship.

Q5. Many-to-many / association class → relations

The Game–Genre many-to-many relationship is converted into a new junction table, GAMEGENRE, containing two foreign keys (GameID, GenreID) that together form a composite primary key. The Member–Game “Rents” many-to-many relationship, together with its association class Rental, is converted into the RENTAL table: RentalID becomes its own primary key (needed because the same member may rent the same game more than once), while MemberID and GameID appear as foreign keys referencing MEMBER and GAME respectively.

Q6. Superclass/subclass mapping

The Member generalization is mapped using the “one table per subclass” approach (class table inheritance). The MEMBER table holds the shared attributes and its own primary key, MemberID. Two additional tables, INDIVIDUALMEMBER and CORPORATEMEMBER, each hold only the attributes specific to that subtype, using MemberID as both their primary key and a foreign key referencing MEMBER.MemberID. This enforces the {complete, disjoint} rule at the data level: every MemberID appears in exactly one of the two subtype tables.

Part 4 – SQL Implementation, Sample Data & Business Queries

4.1 Create Tables

Six related relations were implemented: Publisher, Game, Member, IndividualMember, CorporateMember, and Rental. The script below shows PRIMARY KEY, FOREIGN KEY, NOT NULL, and additional constraints (CHECK, UNIQUE, DEFAULT).

```
CREATE TABLE Publisher (
   PublisherID     INT,
   PublisherName   VARCHAR(100) NOT NULL,
   Country         VARCHAR(50)  NOT NULL,
   YearFounded     INT,
   TargetGroup     VARCHAR(50),
   PRIMARY KEY (PublisherID)
);

CREATE TABLE Game (
   GameID       INT,
   GameName     VARCHAR(100) NOT NULL,
   RentPrice    DECIMAL(10,2) NOT NULL,
   PublisherID  INT NOT NULL,
   PRIMARY KEY (GameID),
   FOREIGN KEY (PublisherID) REFERENCES Publisher(PublisherID),
   CHECK (RentPrice > 0)
);

CREATE TABLE Member (
   MemberID     INT,
   MemberName   VARCHAR(100) NOT NULL,
   PhoneNum     VARCHAR(50) NOT NULL UNIQUE,
   Membership   VARCHAR(20) NOT NULL DEFAULT 'Bronze',
   JoinDate     DATE NOT NULL DEFAULT (CURRENT_DATE),
   ReferredBy   INT NULL,
   PRIMARY KEY (MemberID),
   FOREIGN KEY (ReferredBy) REFERENCES Member(MemberID)
);

CREATE TABLE IndividualMember (
   MemberID       INT,
   IDCardNumber   VARCHAR(20) NOT NULL UNIQUE,
   PRIMARY KEY (MemberID),
   FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
);

CREATE TABLE CorporateMember (
   MemberID       INT,
   CompanyName    VARCHAR(100) NOT NULL,
   TaxID          VARCHAR(20) NOT NULL UNIQUE,
   PRIMARY KEY (MemberID),
   FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
);

CREATE TABLE Rental (
   RentalID       INT,
   MemberID       INT NOT NULL,
   GameID         INT NOT NULL,
   DateRent       DATE NOT NULL,
   DateGiveBack   DATE NULL,
   PRIMARY KEY (RentalID),
   FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
   FOREIGN KEY (GameID) REFERENCES Game(GameID)
);
```

4.2 Sample Data

Each implemented table was populated with realistic sample data: 10 publishers, 12 games, 20 members (10 individual, 10 corporate), and 20 rental transactions with varied dates, some already returned and some still outstanding (DateGiveBack = NULL). Example rows:

```
INSERT INTO Publisher (PublisherID, PublisherName, Country, YearFounded, TargetGroup) VALUES
(1, 'Rockstar Games', 'USA', 1998, 'Adult'),
(4, 'Nintendo', 'Japan', 1889, 'Kids'), ...

INSERT INTO Rental (RentalID, MemberID, GameID, DateRent, DateGiveBack) VALUES
(1, 1, 5, '2024-01-10', '2024-01-15'),
(3, 3, 2, '2024-03-01', NULL), ...
```

(Full INSERT statements for all six tables, at least 10 rows each, are included in the accompanying .sql file: 6810545441_Krittitee_Chaisang_HW6.sql)

4.3 Business Questions and Results

Q1 (JOIN) – Business Question: Which games are currently rented out and not yet returned?

```
SELECT m.MemberName, g.GameName, r.DateRent
FROM Rental r
JOIN Member m ON r.MemberID = m.MemberID
JOIN Game g ON r.GameID = g.GameID
WHERE r.DateGiveBack IS NULL;
```

MemberName	GameName	DateRent
Chaiwat Petch	RedDead Redemption 2	2024-03-01
Kittipong Saelim	Resident Evil 4	2024-05-01
Chaiwat Petch	Assassins Creed Mirage	2024-06-10
Chiang Mai Gaming Hub	Resident Evil 4	2024-07-15

Q2 (GROUP BY + aggregate) – Business Question: Which publisher generates the most rental revenue?

```
SELECT p.PublisherName, COUNT(r.RentalID) AS TotalRentals, SUM(g.RentPrice) AS TotalRevenue
FROM Rental r
JOIN Game g ON r.GameID = g.GameID
JOIN Publisher p ON g.PublisherID = p.PublisherID
GROUP BY p.PublisherName
ORDER BY TotalRevenue DESC;
```

PublisherName	TotalRentals	TotalRevenue
Rockstar Games	7	8500
Activision	2	2200
Capcom	2	1800
Square Enix	1	1200
Nintendo	2	1200
EA Sports	2	1000
CD Projekt	1	950

PublisherName	TotalRentals	TotalRevenue
Ubisoft	1	850
Bandai Namco	1	850
Roblox Corporation	1	67

Q3 (HAVING) – Business Question: Which members are repeat customers (rented more than once)?

```
SELECT m.MemberName, COUNT(*) AS TotalRentals
FROM Rental r
JOIN Member m ON r.MemberID = m.MemberID
GROUP BY m.MemberName
HAVING COUNT(*) > 1;
```

MemberName	TotalRentals
Chaiwat Petch	2
Nattapong Srisuk	3
Thanakorn Meesuk	2
Waraporn Jindarat	2

Q4 (CASE + date function) – Business Question: What is the status and duration of each rental?

```
SELECT r.RentalID, m.MemberName, g.GameName,
    DATEDIFF(IFNULL(r.DateGiveBack, CURDATE()), r.DateRent) AS DaysRented,
    CASE WHEN r.DateGiveBack IS NULL THEN 'Still Rented' ELSE 'Returned' END AS RentalStatus
FROM Rental r
JOIN Member m ON r.MemberID = m.MemberID
JOIN Game g ON r.GameID = g.GameID
ORDER BY DaysRented DESC;
```

RentalID	MemberName	GameName	DaysRented	Status
3	Chaiwat Petch	RedDead Redemption 2	*	Still Rented
8	Kittipong Saelim	Resident Evil 4	*	Still Rented
13	Chaiwat Petch	Assassins Creed Mirage	*	Still Rented
18	Chiang Mai Gaming Hub	Resident Evil 4	*	Still Rented
15	Nattapong Srisuk	GTA6	6	Returned
1	Nattapong Srisuk	Fifa24	5	Returned
2	Waraporn Jindarat	GTA5	5	Returned

** DaysRented for open rentals (DateGiveBack IS NULL) is computed as CURDATE() − DateRent, so it grows with however much time has passed since the rental date and will differ depending on the day the query is run.*

Q5 (own design) – Business Question: Do individual members or corporate members rent more frequently?

```
SELECT 'Individual' AS MemberType, COUNT(DISTINCT r.MemberID) AS ActiveRenters,
```

```
    COUNT(r.RentalID) AS TotalRentals
FROM Rental r
JOIN IndividualMember im ON r.MemberID = im.MemberID
UNION
SELECT 'Corporate' AS MemberType, COUNT(DISTINCT r.MemberID), COUNT(r.RentalID)
FROM Rental r
JOIN CorporateMember cm ON r.MemberID = cm.MemberID;
```

MemberType	ActiveRenters	TotalRentals
Individual	10	15
Corporate	5	5

Individual members are both more numerous among renters and rent more frequently on average (1.5 rentals/member) than corporate members (1.0 rentals/member).